

A Symbolic–Probabilistic AI Agent for Minesweeper

Author(s): L. Leiria

Correspondence: info@dataprecog.com

Abstract

This paper presents a Prolog-based Minesweeper player implemented using symbolic artificial intelligence. The system models the game board as a dynamic knowledge base containing cell states, mine locations, board dimensions, and game status. It combines deterministic rule execution with bounded model enumeration and probability estimation. Deterministic operations support board construction, cell opening, flagging, recursive expansion of empty regions, and win/loss detection. For decision-making, the agent identifies frontier cells adjacent to revealed clues, enumerates legal mine assignments for sufficiently small frontiers, and derives mine probabilities from the resulting models. When exact enumeration becomes computationally expensive, it falls back to local or global probability estimates. The resulting architecture is interpretable and data-independent, but its scalability, probability calibration, and treatment of unconstrained cells impose important limitations. The implementation demonstrates how declarative logic programming can express both the rules of Minesweeper and a practical uncertainty-aware solving strategy.

Keywords: symbolic artificial intelligence, artificial intelligence, constraint satisfaction, probabilistic reasoning, logic programming, model enumeration, Prolog, Minesweeper

1. Introduction

Minesweeper is a partially observable constraint-solving problem. A player must infer the locations of hidden mines from numerical clues revealed on neighboring cells. Each revealed number indicates how many mines occur among its adjacent cells. The player must therefore combine local logical constraints with decisions under uncertainty.

The discussed implementation, `minesweeper.pl`, treats Minesweeper as a symbolic reasoning task. Rather than learning from examples, the program explicitly represents the board and applies rules to derive legal moves. It is implemented in Prolog, a language particularly suited to knowledge representation, relational definitions, recursive search, and constraint-style reasoning.

The system has two complementary objectives:

1. Maintain a logically consistent representation of the game state.
2. Select actions that minimize the estimated probability of opening a mine.

The implementation is notable because it does not rely exclusively on simple local rules. It can enumerate possible mine configurations in a frontier region and use those configurations to estimate

the probability that each hidden cell contains a mine. This makes the player a hybrid symbolic–probabilistic system: its underlying knowledge is symbolic, while its action selection uses probabilities derived from logical models.

2. System Architecture

The program is organized around a dynamic Prolog knowledge base. The main predicates exported by the module are:

```
new_game/4,  
open_cell/2,  
flag_cell/2,  
unflag_cell/2,  
print_board/0,  
print_board/1,  
solve/0,  
solve/1,  
game_status/1
```

The board is represented using several dynamic predicates:

```
board_width/1  
board_height/1  
mine/2  
cell/3  
game_status/1
```

A coordinate is represented as an ordered pair, such as 3-5, corresponding to column 3 and row 5. Cell states are represented symbolically:

- hidden
- open(Number)
- flagged
- exploded

The board dimensions and mine positions are stored as facts. Cell visibility and status are stored independently from mine locations. This separation allows the implementation to simulate a game in which the program has access to the underlying mine layout while restricting its actions to the visible board state.

At initialization, `new_game/4` clears any previous board and creates a new one:

```
new_game(Width, Height, Mines, InitialOpen)
```

The predicate validates the dimensions, records the mine positions, initializes every cell as hidden, and then opens the cells supplied in `InitialOpen`.

3. Formal Model of Minesweeper

Let the board be a rectangular grid

$$B = \{(x,y) \mid 1 \leq x \leq W, 1 \leq y \leq H\}.$$

Let $M \subseteq B$ denote the set of mine locations. For every cell $c \in B$, define its neighborhood $N(c)$ as the set of up to eight horizontally, vertically, and diagonally adjacent cells.

For a non-mine cell c , the clue value is

$$\text{clue}(c) = |N(c) \cap M|.$$

The implementation computes neighborhoods through the `neighbour/4` predicate. It considers all combinations of coordinate offsets

$$\Delta x, \Delta y \in \{-1, 0, 1\},$$

excluding the zero offset (0,0), and retains only coordinates inside the board.

The predicate `neighbours/3` collects all valid neighboring coordinates into a list. This relation is reused by clue computation, flag counting, hidden-cell detection, and constraint validation.

4. Game-State Operations

4.1 Opening a Cell

The `open_cell/2` predicate performs the central game operation. It succeeds only when the game is running and the target cell is hidden.

If the target coordinate is a mine, the cell is replaced with `exploded` and the game status becomes `lost`. Otherwise, the program computes the adjacent mine count and changes the cell state to `open(Count)`.

When the count is zero, the implementation recursively opens neighboring cells:

```
Count := 0
```

This reproduces the automatic expansion found in conventional Minesweeper interfaces. Because each cell is opened only when it is hidden, recursive expansion terminates even when neighboring zero-valued cells form a connected region.

After a successful opening, `check_win/0` is called. The program declares victory when no hidden cells remain:

```
\+ cell(_, _, hidden)
```

This condition means that a game is considered won when every cell has been opened or flagged. In the implementation, however, the win test specifically checks for the absence of hidden cells, so incorrectly flagged cells could potentially contribute to a win if all hidden cells have been removed.

4.2 Flagging and Unflagging

The predicates `flag_cell/2` and `unflag_cell/2` change a cell between the `hidden` and `flagged` states. Flagging does not itself prove that a cell is a mine. It records the agent's current commitment.

The implementation's solver flags only cells assigned probability 1.0. Consequently, flags generated by the solver correspond to cells that are classified as mines under every considered assignment, although the broader game interface permits arbitrary flags.

4.3 Clue and Neighborhood Information

Several predicates derive local information:

- `adjacent_mine_count/3` counts actual mines around a cell.
- `adjacent_flag_count/3` counts flagged neighboring cells.
- `hidden_neighbours/3` identifies hidden neighboring cells.
- `frontier_cells/1` identifies hidden cells adjacent to at least one opened clue.

The distinction between actual mines and flags is important. Clues are generated from the hidden ground-truth board, whereas reasoning constraints use flags as the agent's current mine commitments.

5. Constraint Representation

For each opened cell c with clue value k , the program constructs a constraint over neighboring frontier cells. If $f(c)$ is the number of flagged neighbors and $H(c)$ is the set of hidden neighbors, then the required number of mines among the still-hidden neighbors is

$$r(c) = k - f(c).$$

For a candidate assignment A of mines to frontier cells, the assignment is legal if

$$|A \cap N(c)| = r(c)$$

for every opened clue cell c .

This condition is implemented by `clue_is_satisfied/3`. The predicate counts candidate mine positions adjacent to each clue and compares that count with the number still required after subtracting flagged neighbors.

The complete assignment is accepted only if every opened clue is satisfied:

```
forall(  
    cell(X, Y, open(Number)),  
    clue_is_satisfied(X, Y, Number, Assignment)  
)
```

This universal condition gives the solver a declarative definition of consistency. A candidate configuration is not evaluated by simulating moves; it is tested directly against all currently visible clues.

6. Frontier-Based Reasoning

The frontier is the set of hidden cells adjacent to at least one revealed clue. These cells are the most directly constrained portion of the board.

The implementation computes the frontier by collecting hidden neighbors of every open cell and removing duplicates. Let F denote the frontier. The solver focuses exact reasoning on F , rather than enumerating all hidden cells on the board.

This is a useful decomposition. Hidden cells far from revealed clues cannot usually be distinguished using local evidence. By restricting exhaustive search to the frontier, the program reduces the size of the search space from

$$2^{|H|}$$

for all hidden cells H , to

$$2^{|F|}$$

for the constrained region.

However, this reduction is effective only when the frontier is substantially smaller than the entire hidden region.

7. Exact Frontier Enumeration

When the frontier contains no more than 18 cells, the solver invokes:

```
enumerate_frontier(Frontier, Assignments)
```

The predicate `binary_assignment/2` recursively generates every subset of the frontier. Each subset represents the cells assumed to contain mines. Consequently, a frontier with n cells generates at most

$$2^n$$

candidate assignments.

Each candidate is filtered through `assignment_satisfies_clues/1`. The surviving assignments represent all frontier mine configurations consistent with the currently visible clues and existing flags.

For a frontier cell x , the estimated mine probability is

$$P(x \text{ is a mine}) = \frac{\text{number of legal assignments containing } x}{\text{total number of legal assignments}}$$

This is a model-counting interpretation of probability. The system assumes that each legal assignment is equally likely. Under that assumption, the frequency with which a cell appears as a mine provides its probability estimate.

For example, suppose four legal assignments exist and a particular cell appears as a mine in one of them. Its estimated mine probability is

$$P=1/4=0.25.$$

A cell appearing in every legal assignment receives probability 1.0, while a cell appearing in none receives probability 0.0.

8. Treatment of Unconstrained Cells

Not all hidden cells necessarily belong to the frontier. For cells outside the frontier, the implementation computes an average probability based on the remaining mine count.

For each legal frontier assignment A_i , the number of mines assigned to the frontier is $|A_i|$. The average number of assigned frontier mines is

$$m = \frac{1}{|A|} \sum_{i=1}^{|A|} |A_i|$$

If T is the total number of mines and F is the number of already flagged cells, the estimated number of mines remaining outside the frontier is

$$R = T - F - m.$$

If U is the number of unconstrained hidden cells, the probability assigned to each unconstrained cell is

$$P_{unconstrained} = \frac{R}{U}$$

The value is clamped to the interval $[0,1]$. This approach preserves an approximate global mine-count constraint while treating unconstrained cells symmetrically.

9. Approximation Strategies

Exact enumeration is used only when the frontier has at most 18 cells. For larger frontiers, the program prints a diagnostic message and uses local probabilities instead.

For a hidden cell x , each adjacent open clue provides a local estimate. If the clue value is k , the number of adjacent flags is f , and the number of hidden neighbors is h , then the local estimate is

$$p = \frac{k-f}{h}$$

The program averages the estimates obtained from all adjacent clues:

$$P(x) = \frac{1}{q} \sum_{j=1}^q P_j,$$

where q is the number of neighboring clues that provide an estimate.

If no neighboring clue is available, the implementation assigns a default probability of 0.5. This value is a neutral fallback rather than a probability derived from the board's remaining mine count.

When no frontier exists, the solver uses a global fallback:

$$P(x) = \frac{T-F}{|H|},$$

where H is the set of hidden cells. This assumes that remaining mines are uniformly distributed among all hidden cells.

These approximations allow the solver to continue operating on larger or less informative boards, but they do not generally produce probabilities that are mathematically consistent with all board constraints.

10. Action Selection

The solver repeatedly performs the following cycle:

1. Calculate probabilities for hidden cells.
2. Flag any cell with probability 1.0.
3. Otherwise, open any cell with probability 0.0.
4. Otherwise, open the cell with the smallest estimated mine probability.
5. Print the selected action and updated board.
6. Continue until the game ends or a step limit is reached.

The decision policy is encoded in `choose_move/2`. It gives priority to certain mine conclusions, then certain safe moves, and finally minimum-risk moves.

The policy can be expressed as:

$$A^* = \begin{cases} \text{flag}(x) & \text{if } P(x)=1, \\ \text{open}(x) & \text{if } P(x)=0, \\ \text{open}(\arg\min_x P(x)) & \text{otherwise.} \end{cases}$$

The sorting procedure uses probability as the primary key and coordinates as deterministic tie-breakers. This ensures reproducible behavior when multiple cells have equal estimated probabilities.

11. Computational Complexity

Let f be the number of frontier cells. Frontier enumeration requires generating up to 2^f assignments. For each assignment, the implementation checks every open clue and counts assigned neighboring mines. If C is the number of open clues and d is the maximum neighborhood size, the rough worst-case cost is

$$O(2^f C d) .$$

Because Minesweeper neighborhoods contain at most eight cells, d is bounded by a small constant. The exponential term 2^f therefore dominates the exact solver.

At the configured threshold $f \leq 18$, the maximum number of candidate assignments is

$$2^{18} = 262,144.$$

This is manageable for small boards but can become expensive when repeated across many solver steps. The threshold is consequently a practical engineering trade-off between exact reasoning and computational cost.

The local-probability method is substantially cheaper. It processes hidden cells and their neighboring clues without enumerating assignments, producing a complexity that is approximately linear in the number of relevant cell–clue relationships.

12. Interpretation as Symbolic Artificial Intelligence

The implementation exemplifies several characteristics of symbolic AI.

First, knowledge is explicitly represented. Board dimensions, mine positions, cell states, and game status are all encoded as logical facts.

Second, the system reasons through rules. For example, a clue is satisfied only when the number of assigned neighboring mines equals the required count. This rule is transparent and inspectable.

Third, inference is explainable. A probability can be traced to the set of legal assignments that support or exclude a cell as a mine. A flag with probability 1.0 means that every surviving model assigns that cell as a mine.

Fourth, the system does not require training data. Its behavior is determined by the game rules and the search strategy.

The probabilistic component does not replace symbolic reasoning. Instead, it aggregates the results of symbolic model enumeration. In this sense, the program is a symbolic constraint solver with a probabilistic decision layer.

13. Strengths

The implementation has several important strengths.

Interpretability. The representation of cells, clues, flags, and assignments is explicit. The solver's decisions can be inspected in terms of board facts and logical constraints.

Correct local game mechanics. The program supports mine detection, recursive opening of empty regions, flagging, unflagging, win detection, and loss detection.

Exact reasoning for small frontiers. When the frontier is sufficiently small, the solver considers every binary assignment and rejects those that violate the visible clues.

Use of global mine information. The probability assigned to unconstrained cells attempts to account for the total number of mines and the number already flagged.

Graceful degradation. When exact enumeration is too expensive, the system switches to local or global estimates instead of terminating.

Deterministic behavior. Coordinate-based tie-breaking makes repeated runs reproducible for the same initial state.

14. Limitations

14.1 Exponential Enumeration

The principal limitation is the exponential complexity of frontier enumeration. Increasing the frontier by one cell doubles the number of candidate assignments. The fixed threshold of 18 cells

prevents unbounded computation but also causes the solver to abandon exact reasoning on moderately complex positions.

14.2 Equal-Likelihood Assumption

The exact probabilities assume that all legal assignments are equally likely. This is a reasonable default when no additional prior information is available, but it is not always justified. Different assignments may have different probabilities under a specified game-generation process or first-move policy.

14.3 Approximate Local Probabilities

The local method averages ratios from neighboring clues independently. It does not enforce consistency between those clues. As a result, the estimated probabilities may not correspond to any globally valid mine distribution.

14.4 Default Probability of 0.5

A hidden cell without an applicable local estimate receives probability 0.5, regardless of the number of remaining mines or hidden cells. This can be inconsistent with the known mine count. A count-based fallback would generally be more principled.

14.5 Potential Effects of Incorrect Flags

Flags are treated as known mines when evaluating clues. If a flag is incorrect, subsequent reasoning can become inconsistent. The solver itself flags only probability-one cells, which reduces this risk, but external calls to `flag_cell/2` can introduce invalid assumptions.

14.6 Win Condition Semantics

The win test checks whether any hidden cells remain. It does not explicitly verify that every flagged cell is a mine. Depending on the intended game semantics, a stronger condition would require all non-mine cells to be open and all mines either flagged or otherwise accounted for.

14.7 Ground-Truth Mine Access

The implementation stores all mine locations in the `mine/2` predicate. This is suitable for simulation and testing, but it means that the environment and game engine share the same knowledge base. The solver does not infer mine locations from an external perceptual interface; it operates over a simulated board whose hidden state is available to the game mechanics.

16. Conclusion

This paper describes the implementation of *A Symbolic-Probabilistic AI Agent for Minesweeper* based on symbolic state representation, logical constraint checking, bounded model enumeration, and probability-guided action selection. Its central contribution is the conversion of visible clues into a set of legal frontier mine assignments. Probabilities are then obtained by counting how frequently each cell appears as a mine across those assignments.

The architecture demonstrates the practical value of combining exact symbolic inference with computationally bounded approximation. Exact enumeration provides strong conclusions for small

frontier regions, while local and global heuristics allow the solver to continue operating when exhaustive reasoning becomes expensive.

The system remains limited by exponential search, simplifying assumptions about assignment probabilities, and approximate treatment of large frontiers. Nevertheless, it provides a compact and instructive example of declarative artificial intelligence: the rules of the environment, the constraints governing valid states, and the decision policy are all represented explicitly in executable logic.

Source Artifact

This discussed “*Symbolic–Probabilistic AI Agent for Minesweeper*” implementation can be found at https://github.com/dataprecog/MinesweeperPlayer_AI/tree/main/src